process inside a container with a UID that happens to correspond to an existing user in the host OS, then the container process will inherit all the permissions of that user on the host. Normally, this isn't a problem because the process inside the container cannot see or access any files or other resources on the host unless they are explicitly shared, but for added security your Docker images can create a new user and group and let the kernel assign an unused UID and GID without any existing permissions in the host OS. If an attacker manages to exploit a vulnerability to gain access to the host OS or filesystem, they will have no (or very limited) permissions.
You should avoid running containers as the root user because root can often break out of the container and run commands directly on the host with no restrictions. If you need to run a container as root, then you can use a Linux *user namespace* to map UIDs within the container to a different range of UIDs on the host. This allows a process running as UID 0 (root) within a container to be mapped to a non-privileged UID such as 20000 in the host. As far as the container is concerned, the process is running as root, but it would not have root privileges if it ever could break out of the container to access the host. See https://docs.docker.com/engine/security/userns-remap/ for how to enable a user namespace in Docker. This is not yet possible in Kubernetes, but there are several alternative options for reducing user privileges inside a pod that are discussed later in the chapter.

When you build a Docker image it gets cached by the Docker daemon that runs the build process. To use the image elsewhere, such as within a Kubernetes cluster, you must first push the image to a container repository such as Docker Hub or a private repository within your organization. To avoid having to configure a repository and credentials in this chapter, you can instead build directly to the Docker daemon used by minikube by running the following commands in your terminal shell. You should specify version 1.16.2 of Kubernetes to ensure compatibility with the examples in this book. Some of the examples require minikube to be running with at least 4GB of RAM, so use the `--memory` flag to specify that.

```
minikube start \
    --kubernetes-version=1.16.2 \    #A
```